CreditRisk AI: Machine Learning-Based Credit Card Approval Prediction System

Project Description:

CreditRisk AI harnesses classical Machine Learning to provide an intelligent credit-risk assessment experience, offering financial institutions a fast, transparent, and auditable way to screen credit-card applicants. The platform includes a Data Pipeline module, which cleans and de-duplicates applicant records and engineers financial ratio features; a Model Comparison module, which benchmarks Logistic Regression, Decision Tree, Random Forest, and XGBoost via stratified cross-validation; and a Risk Prediction module, which returns an approve/reject decision together with a calibrated risk score for any applicant profile.

Utilizing a single scikit-learn Pipeline that bundles preprocessing (imputing, scaling, one-hot encoding) with the tuned classifier, CreditRisk AI processes applicant inputs to deliver a consistent prediction, whether the request comes from the training notebook or the live app. Built with Python and deployed publicly via Gradio's share link, the platform ensures a seamless and user-friendly experience. With honest handling of class imbalance, a data-driven decision threshold, and transparent reporting of model limitations, CreditRisk AI empowers analysts, students, and recruiters reviewing the project to understand exactly how and why a decision was made.

Scenarios

Scenario 1: A young applicant describes themselves as a Pensioner with only two years of employment history and a modest income. The system flags an unusual combination against the patterns it learned from historical applicants, returns a high risk score, and the application is rejected.

Scenario 2: A mid-career State Servant with stable long-term employment, a matching age/experience profile, and moderate income applies. CreditRisk AI recognizes this as consistent with historically low-risk applicants, returns a low risk score, and the application is approved.

Scenario 3: An analyst wants to understand borderline cases before approving a loan product change. They enter a profile with average income and family size; CreditRisk AI returns a risk score close to the decision threshold, illustrating to the analyst exactly how sensitive the outcome is and why threshold selection (not just the raw model) matters for real decisions.

Technical Architecture:

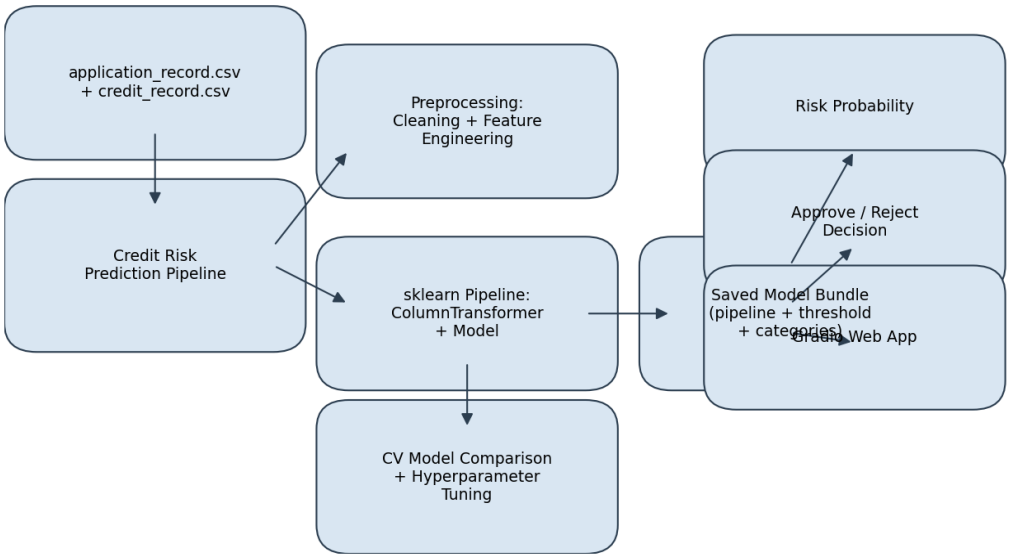

Figure 1: CreditRisk AI system architecture

Description: CreditRisk AI utilizes a modular pipeline architecture to deliver reproducible, ML-driven risk decisions. The data layer merges applicant demographics with derived repayment-history labels. A single scikit-learn ColumnTransformer + model Pipeline handles all preprocessing and inference, so the exact transformation used at training time is guaranteed at prediction time. The saved pipeline is loaded by a Gradio front end, which collects applicant details and displays the risk score and decision, with the app exposed publicly via Gradio's built-in share tunnel.

(Note: this project does not use a relational database — applicant data is processed in-memory as pandas DataFrames — so no ER diagram is required. If a database were added for storing submitted applications, an ER diagram and description would be included here.)

Pre-requisites:

- Python Programming Proficiency: Python Documentation
- scikit-learn Familiarity: scikit-learn Documentation
- Pandas & NumPy for Data Handling: Pandas Documentation
- Imbalanced Classification Concepts: imbalanced-learn Documentation
- XGBoost Familiarity: XGBoost Documentation
- Gradio for Model Deployment: Gradio Documentation
- Version Control with Git: Git Documentation
- Google Colab / Jupyter Notebook Environment: Colab Documentation

Project Workflow:

Milestone 1: Model Selection and Architecture

- Activity 1.1: Load application_record.csv and credit_record.csv and explore their structure.
- Activity 1.2: Research and select the appropriate classification models for tabular, imbalanced credit-risk data.
- Activity 1.3: Define the architecture of the application, detailing interactions between the preprocessing pipeline, model, and Gradio front end.
- Activity 1.4: Set up the development environment, installing necessary libraries in Google Colab.

Milestone 2: Core Data Processing & Feature Engineering

- Activity 2.1: Clean the data, handle missing occupation values, and remove duplicate applications.
- Activity 2.2: Engineer features (age, years employed, family size, income ratios) and construct the TARGET label from credit_record.csv.

Milestone 3: Model Training & Pipeline Development

- Activity 3.1: Build the ColumnTransformer preprocessing pipeline and compare models with stratified cross-validation.
- Activity 3.2: Compare class-weighting vs. SMOTE, then tune the winning model with RandomizedSearchCV.

Milestone 4: Evaluation & Frontend Development

- Activity 4.1: Evaluate the tuned model (ROC-AUC, confusion matrix, feature importance) and select a decision threshold using Youden's J statistic.
- Activity 4.2: Design and develop the Gradio user interface for real-time predictions.

Milestone 5: Deployment

- Activity 5.1: Save the trained pipeline, threshold, and category metadata as a single model bundle.
- Activity 5.2: Deploy the application publicly using Gradio's share=True tunnel to expose a secure public URL.

Milestone 6: Conclusion

Milestone 1: Model Selection and Architecture

In this milestone, we focus on selecting the appropriate classical ML approach for the credit-risk prediction task. This involves researching how different model families handle imbalanced, mixed categorical/numeric tabular data, and ensuring the chosen approach aligns with the project's goal of producing an honest, well-calibrated risk score rather than an inflated accuracy figure.

Activity 1.1: Load and Explore the Datasets

Before any modeling can begin, the two source files need to be loaded and understood: application_record.csv (applicant demographics and financials) and credit_record.csv (month-by-month repayment status per applicant).

```
app = pd.read_csv("application_record.csv")
credit = pd.read_csv("credit_record.csv")

print("application_record:", app.shape)
print("credit_record     :", credit.shape)
app.head()
```

Activity 1.2: Research and Select the Appropriate Model(s)

Here are the things that needed to be researched to select the best model(s) for the project:

- Understand the Project Requirements: Review the specific needs of CreditRisk AI, focusing on an imbalanced binary classification problem (high risk vs. low risk applicant).
- Explore scikit-learn's & XGBoost's Model Documentation: Examine available classifiers, their handling of categorical data, class weighting options, and typical performance on tabular data.
- Evaluate Model Performance: Compare Logistic Regression, Decision Tree, Random Forest, and XGBoost using stratified 5-fold cross-validation scored on ROC-AUC (not accuracy, which is misleading on an imbalanced target).
- Select the Optimal Model: Choose the model with the highest mean cross-validated ROC-AUC, then tune it further with RandomizedSearchCV, rather than assuming the most complex model automatically wins.

Activity 1.3: Define the Architecture of the Application

- Draft an Architectural Diagram: Create a visual representation of the pipeline architecture, including data ingestion, preprocessing, model, and the Gradio front end (see Figure 1).
- Detail Frontend Functionality: Outline how users interact with the application: entering applicant details (income, employment, family status, etc.) and clicking Predict.
- Outline Backend Responsibilities: Specify how the saved pipeline validates and transforms user input, computes derived features (age, years employed, income ratios), and produces a probability.
- Describe Model Integration Points: Define how build_applicant_row() assembles a single-row DataFrame, how the pipeline's ColumnTransformer encodes it identically to training time, and how the tuned threshold converts a probability into an approve/reject decision.

Activity 1.4: Set Up the Development Environment

- Install Python and Pip: Use Google Colab, which ships with Python 3.10+ and pip pre-installed.
- Install Required Libraries:

```
!pip install -q xgboost imbalanced-learn shap gradio joblib
```

- Set Up Application Structure: Organize the Colab notebook into clearly labeled sections for data loading, cleaning, EDA, modeling, evaluation, saving, and the Gradio app — mirroring the milestone structure of this document.

Milestone 2: Core Data Processing & Feature Engineering

Activity 2.1: Clean the Data

OCCUPATION_TYPE is missing for applicants without a listed occupation — fill with an explicit "Unknown" category rather than dropping rows. Remove duplicate applicants, not just duplicate rows: because ID is unique per row by construction, a plain drop_duplicates() never removes anything, so duplicates must be dropped on every column except ID.

```
app["OCCUPATION_TYPE"] = app["OCCUPATION_TYPE"].fillna("Unknown")

before = app.shape[0]
app = app.drop_duplicates(subset=app.columns.difference(["ID"])).reset_index(drop=True)
after = app.shape[0]

print(f"Duplicate applications removed: {before - after}")
```

Activity 2.2: Feature Engineering & Target Construction

DAYS_EMPLOYED == 365243 is a placeholder used for pensioners/unemployed applicants and is treated as missing rather than as "employed for 1000 years". DAYS_BIRTH / DAYS_EMPLOYED are converted into readable ages/years, and ratio features (Family_Size, Income_Per_Family, Income_Per_Year_Employed) are engineered.

```
app["DAYS_EMPLOYED_ANOMALY"] = (app["DAYS_EMPLOYED"] == 365243).astype(int)
app["DAYS_EMPLOYED"] = app["DAYS_EMPLOYED"].replace(365243, np.nan)

app["AGE_YEARS"] = (app["DAYS_BIRTH"].abs() / 365.25).round(1)
app["YEARS_EMPLOYED"] = (app["DAYS_EMPLOYED"].abs() / 365.25).round(1).fillna(0)

app["Family_Size"] = app["CNT_CHILDREN"] + app["CNT_FAM_MEMBERS"]
app["Income_Per_Family"] = app["AMT_INCOME_TOTAL"] / app["Family_Size"].replace(0, 1)
```

The TARGET label is built from credit_record.csv: an applicant is labeled high risk (TARGET = 1) if they were ever 30+ days overdue (STATUS in 1,2,3,4,5), and low risk otherwise.

```
def is_high_risk(status):
    return 1 if status in ["1", "2", "3", "4", "5"] else 0

credit["TARGET"] = credit["STATUS"].apply(is_high_risk)
credit_group = credit.groupby("ID").agg(TARGET=("TARGET", "max")).reset_index()
final_df = app.merge(credit_group, on="ID", how="inner")
```

Milestone 3: Model Training & Pipeline Development

Milestone 3 focuses on building the reusable preprocessing pipeline and training logic that serves as the backbone of CreditRisk AI. In this milestone, each modeling decision is validated through cross-validation rather than a single train/test split, and the pipeline is designed so training-time and inference-time transformations can never drift out of sync.

Activity 3.1: Build the Preprocessing Pipeline

```
preprocess = ColumnTransformer(transformers=[
    ("num", Pipeline([
        ("impute", SimpleImputer(strategy="median")),
        ("scale", StandardScaler()),
    ]), num_cols),
    ("cat", Pipeline([
        ("impute", SimpleImputer(strategy="most_frequent")),
        ("ohe", OneHotEncoder(handle_unknown="ignore")),
    ]), cat_cols),
])
```

Activity 3.2: Compare Models and Tune Hyperparameters

- Compare Logistic Regression, Decision Tree, Random Forest, and XGBoost via 5-fold stratified cross-validation on ROC-AUC.
- Compare class_weight/scale_pos_weight against SMOTE oversampling for the leading model, applying SMOTE only within each training fold to avoid data leakage.
- Tune the winning model + imbalance strategy with RandomizedSearchCV, still scored on ROC-AUC.

```
skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=RANDOM_STATE)

search = RandomizedSearchCV(
    final_pipeline_template,
    param_distributions=param_distributions,
    n_iter=15, scoring="roc_auc", cv=skf,
    random_state=RANDOM_STATE, n_jobs=-1, refit=True,
)
search.fit(X_train, y_train)
best_pipeline = search.best_estimator_
```

Milestone 4: Evaluation & Frontend Development

Activity 4.1: Evaluate the Model and Tune the Decision Threshold

The tuned pipeline is evaluated on a held-out test set using ROC-AUC, precision, recall, F1, and a confusion matrix. Rather than using the default 0.5 cutoff, the decision threshold is selected using Youden's J statistic (J = TPR - FPR) off the ROC curve, which produces a far more balanced, usable operating point than either the default cutoff or a naive F1-maximizing threshold on this genuinely weak-signal dataset.

```
fpr, tpr, roc_thresholds = roc_curve(y_test, test_proba)
youden_j = tpr - fpr
best_idx = np.argmax(youden_j)

OPTIMAL_THRESHOLD = float(roc_thresholds[best_idx])
print("Optimal threshold (Youden's J):", round(OPTIMAL_THRESHOLD, 3))
```

Feature importances are also extracted from the winning model to identify which applicant attributes drive its predictions, giving reviewers transparency into the model's reasoning.

Activity 4.2: Design and Develop the Gradio User Interface

- Set Up the Input Form: Build a Gradio Blocks layout with fields for gender, car/realty ownership, income, income type, education, family status, housing type, occupation, age, years employed, and family members.
- Pull Dropdown Options from the Model Bundle: Populate every dropdown directly from the exact category values seen during training (saved in the bundle), so the UI can never drift out of sync with the model again.
- Display Results Dynamically: On clicking Predict, show the approve/reject decision, the risk score, and the decision threshold used, formatted as Markdown.

```
def predict_credit(gender, own_car, own_realty, children, income, income_type,
                   education, family_status, housing_type, occupation,
                   age_years, years_employed, family_members):
    x = build_applicant_row(gender, own_car, own_realty, children, income,
                            income_type, education, family_status,
                            housing_type, occupation, age_years,
                            years_employed, family_members)
    risk_prob = pipeline.predict_proba(x)[0, 1]
    approved = risk_prob < THRESHOLD
    ...
```

Milestone 5: Deployment

In Milestone 5, the focus is on saving the trained pipeline as a single reusable artifact and deploying the Gradio application publicly so it can be shared and demoed without any local installation on the viewer's side.

Activity 5.1: Save the Model Bundle

```
bundle = {
    "pipeline": best_pipeline,
    "threshold": OPTIMAL_THRESHOLD,
    "model_name": best_model_name,
    "num_cols": num_cols,
    "cat_cols": cat_cols,
    "categories": {c: sorted(X[c].dropna().unique().tolist()) for c in cat_cols},
}

joblib.dump(bundle, "credit_model_bundle.pkl")
```

Activity 5.2: Public Deployment via Gradio

Gradio's built-in share tunnel creates a secure, temporary public URL for the local app, similar in role to Ngrok, without requiring a separate cloud hosting provider.

```
app_ui.launch(share=True, debug=False)
```

- Once running, Gradio prints a public URL such as https://xxxxxxx.gradio.live in the notebook output.
- Share this URL with anyone who can access the CreditRisk AI app directly from their browser, without any installation.

Important Notes:

- Gradio's free public link is temporary and expires when the Colab runtime is closed or after a period of inactivity — rerun the launch cell to get a new URL.
- For a persistent deployment, the saved credit_model_bundle.pkl can be wrapped in a small Flask/FastAPI service and deployed to a cloud host.
- Never commit the model bundle's underlying raw data files to a public repository if they contain sensitive applicant information.

Exploring the Application's Screens:

Home / Prediction Page:

Description: The single-page Gradio interface of CreditRisk AI serves as both the input form and the prediction tool. It features a clean two-column layout, a header reading "Credit Card Approval Prediction System", and an input panel on the left with the output panel on the right — all on one seamless page.

Input Section:

Description: A form containing all applicant fields: gender, car/realty ownership (radio buttons); income, children, and family members (numeric inputs); income type, education, family status, housing type, and occupation (dropdowns pulled live from the training data's category values); and age and years employed (sliders). A single Predict Credit Status button triggers the model.

Results Section:

Description: Revealed immediately after clicking Predict, this panel displays a clear Approved or Rejected verdict, the underlying risk score as a percentage, the decision threshold it was compared against, and a plain-language confidence statement — giving the reviewer full transparency into why the model reached its decision, not just a bare yes/no.

Conclusion:

CreditRisk AI is a classical Machine Learning platform built with scikit-learn, XGBoost, and imbalanced-learn, and deployed with a clean Gradio interface. It de-duplicates and engineers features from real applicant and credit-history data, compares four model families honestly via stratified cross-validation, handles class imbalance through both class-weighting and SMOTE, tunes hyperparameters with RandomizedSearchCV, and selects a data-driven decision threshold using Youden's J statistic rather than an arbitrary 0.5 cutoff. The project, which included model selection, rigorous evaluation, and public deployment via Gradio, highlights how a disciplined, transparent ML workflow — one that reports honest performance limitations rather than an inflated accuracy figure — can still deliver a genuinely useful decision-support tool. Looking ahead, CreditRisk AI offers opportunities for expansion, such as incorporating external bureau credit-score data for stronger predictive signal, adding SHAP-based per-applicant explanations, and persisting submitted applications to a database for audit and monitoring. By continuously refining the feature set and monitoring the model against new data, the platform is well-positioned to evolve from a portfolio project into a production-grade credit-risk screening assistant.